

Python Fundamentals for Data Analytics – Question Bank

Introduction

This notebook is a structured collection of Python fundamentals built specifically for Data Analytics.

The objective of this question bank is to strengthen Python concepts through a practical question-and-answer approach. Each question focuses on understanding not only the output of the code but also the reasoning behind it.

The topics covered include variables, operators, strings, lists, tuples, dictionaries, sets, conditional statements, loops, functions, list comprehensions, string manipulation, and advanced list operations.

By working through these questions sequentially, learners can build a strong foundation in Python and develop the problem-solving skills required for data analysis and real-world projects.

-- Question

What is the output of the following code?

```
#Q1
a = 25
b = 10

print (a+b)
35

#Q2
print(a-b)
15

#Q3
print(a*b)
250

#Q4
print(a/b)
2.5
```

```
#Q 5
print(a//b)

# // is floor division.
# It removes the decimal part
2

# Q6
print(a % b)
5
```

% is called the modulus operator.

It returns the remainder after division.

```
# Q7
a = 2

print( a**3 )

8
```

** is the power (exponent) operator.

```
# Q8
x = 10
print(x)

print(x+5)

10
15

# Q9

a = 5
a = 10

print(a)

10
```

Python reads from top to bottom.

- The second assignment overwrites the first value.

```
# Q10
```

```
a = 5
a = a + 3
print(a)
8
```

Section 2: Strings

What You Will Learn

In this section, we will learn:

What strings are

- How to store text in variables
- String concatenation
- String indexing
- Positive and negative indexing
- String functions such as len(), upper(), and lower()
- String repetition
- Why Is This Important for Data Analysts?

Most real-world datasets contain text columns such as:

- Customer Name
- City
- State
- Product Category
- Comments
- Email Address

As a Data Analyst, you will frequently:

- Clean text data
- Convert text to uppercase or lowercase
- Extract specific characters
- Calculate string lengths
- Standardize inconsistent values

Examples:

- Bangalore
- bangalore
- BANGALORE

-- These all represent the same city but need to be standardized for accurate analysis.

- Understanding strings is one of the foundational skills for data cleaning and exploratory data analysis (EDA).

Q11

```
name = "Shubham"  
print(name)
```

Shubham

A string is a sequence of characters enclosed in quotes.

12

```
first_name = "Shubham"  
last_name = "Kulkarni"  
  
print(first_name + last_name)
```

ShubhamKulkarni

#Q 13

```
first_name = "Shubham"  
last_name = "Kulkarni"  
  
print(first_name + " " + last_name)
```

Shubham Kulkarni

" " represents a space. Python joins all three strings together.

Q14

```
name = "Python"  
  
print(len(name))
```

6

len() counts the number of characters in a string.

#Q15

```
name = "Python"  
print(name[0])
```

P

String indexing starts at 0. The first character of "Python" is P.

Q 16

```
name = "Python"
```

```
print(name[3])
```

```
h
```

- Python
 - 0 1 2 3 4 5
- #Character at index 3 is h.

```
# Q 17
```

```
name = "Python"
```

```
print(name[-1])
```

```
n
```

Negative indexing starts from the end. -1 refers to the last character.

```
# Q18
```

```
text = "data analyst"
```

```
print(text.upper())
```

```
DATA ANALYST
```

.upper() converts all letters to uppercase.

```
# Q19
```

```
text = "DATA ANALYST"
```

```
print(text.lower())
```

```
data analyst
```

```
# Q20
```

```
text = "python"
```

```
print(text * 3)
```

```
pythonpythonpython
```

Multiplying a string repeats it the specified number of times.

Section 3: Lists

What You Will Learn

In this section, you will learn:

- Creating lists
- Accessing list elements

- Negative indexing
- Adding elements
- Removing elements
- Updating values
- Finding list length
- Sorting lists
- Why Is This Important for Data Analysts?
- Lists are one of Python's most commonly used data structures. They store multiple values in a single variable and form the foundation for working with arrays, datasets, and Pandas DataFrames.

Q21

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits)  
['Apple', 'Banana', 'Mango']
```

Lists store multiple values inside square brackets [].

Q22

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits[0])  
Apple
```

List indexing starts at 0. The first element is "Apple".

Q23

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits[2])  
Mango
```

#Q24

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits[-1])  
Mango
```

-1 refers to the last element of the list.

#Q25

```
fruits = ["Apple","Banana"]
fruits.append("Mango")

print(fruits)

['Apple', 'Banana', 'Mango']
```

append() adds an element to the end of a list.

#Q26

```
numbers = [10,20,30]
print(len(numbers))

3
```

len() returns the number of elements in the list.

#Q27

```
numbers = [10,20,30]
numbers[1] = 50
```

```
print(numbers)

[10, 50, 30]
```

Q28

```
numbers = [10,20,30]
numbers.remove(20)
```

```
print(numbers)

[10, 30]
```

remove() deletes the specified value from the list.

Q29

```
numbers = [30,10,20]
```

```
numbers.sort()
```

```
print(numbers)

[10, 20, 30]
```

Q30

```
numbers = [10,20,30]
numbers.append(40)
```

```
print(numbers)
print(len(numbers))

[10, 20, 30, 40]
4
```

After appending 40, the list contains:

```
[10, 20, 30, 40]
```

The list now has 4 elements.

```
numbers = [10, 20, 30]
numbers.append(40)
print(numbers)
[10, 20, 30, 40]
# Example
numbers = [10, 20, 30]
numbers.insert(1, 15)
print(numbers)
[10, 15, 20, 30]
# What if I want to add at the beginning?
numbers = [10, 20, 30]
numbers.insert(0, 5)
print(numbers)
[5, 10, 20, 30]
# What if I want to replace a value instead of adding?
# Use indexing:
numbers = [10, 20, 30]
numbers[1] = 15
print(numbers)
[10, 15, 30]
#Q 31
```



```

numbers = [10,20,30,40]
print(numbers[1:3])
[20, 30]

# Q32
numbers = [10, 20, 30, 40]
print(numbers[:2])
[10, 20]

# If the start index is omitted, Python starts from the beginning.
numbers[:2]

# means:
numbers[0:2]
[10, 20]

# Q33
numbers = [10, 20, 30, 40]
print(numbers[2:])
[30, 40]

# Q34
numbers = [10, 20, 30, 40]
print(numbers[-2:])
[30, 40]

# Q35
numbers = [10, 20, 30]
numbers.pop()
print(numbers)
[10, 20]

```

Section 4: Tuples

What You Will Learn

In this section, you will learn:

- Creating tuples
- Accessing tuple elements
- Tuple indexing
- Tuple length
- Difference between lists and tuples
- Why Is This Important for Data Analysts?
- Tuples are similar to lists, but they are immutable, meaning their values cannot be changed after creation.

While lists are used more often, tuples appear frequently in:

- Function outputs
- Data records
- Dictionary keys
- SQL query results

```
# Q36
numbers = (10, 20, 30)

print(numbers)

(10, 20, 30)
```

A tuple stores multiple values using parentheses ().

```
#Q 37
numbers = (10, 20, 30)

print(numbers[0])

10

#Q 38
numbers = (10, 20, 30)

print(numbers[-1])

30

# Q39
numbers = (10, 20, 30)
```

```
print(len(numbers))
```

```
3
```

```
#Q40 numbers = (10, 20, 30)
```

```
numbers[1] = 50
```

```
print(numbers)
```

Tuples are immutable.

Once created, their values cannot be modified.

Section 5: Dictionaries

What You Will Learn

In this section, you will learn:

- Creating dictionaries
- Accessing values using keys
- Adding new key-value pairs
- Updating values
- Removing values *Viewing keys and values

Why Is This Important for Data Analysts?

- Dictionaries store data as key-value pairs.

```
#Q 41
```

```
employee = {  
    "Name": "Shubham",  
    "Age": 26  
}
```

```
print(employee)
```

```
{'Name': 'Shubham', 'Age': 26}
```

```
#Q 42
```

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}
```

```
}  
  
print(employee["Name"])  
  
Shubham
```

Python returns the value associated with that key.

```
# Q43  
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
print(employee["Age"])  
  
28
```

The key "Age" stores the value 28

```
# Q44  
  
employee = {  
    "Name": "Shubham"  
}  
  
employee["Age"] = 28  
  
print(employee)  
  
{'Name': 'Shubham', 'Age': 28}
```

A new key-value pair is added to the dictionary.

```
# Q45  
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
employee["Age"] = 29  
  
print(employee)  
  
{'Name': 'Shubham', 'Age': 29}
```

The key "Age" already exists.

Python updates its value from 28 to 29.

Q46

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
print(len(employee))  
2
```

The dictionary contains 2 key-value pairs.

Q47

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
print(employee.keys())  
dict_keys(['Name', 'Age'])
```

.keys() returns all keys in the dictionary.

Q48

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
print(employee.values())  
dict_values(['Shubham', 28])
```

.values() returns all values in the dictionary.

Q49

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
employee.pop("Age")  
  
print(employee)
```

```
{'Name': 'Shubham'}
```

pop() removes the specified key and its value.

Q50

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
print("Department" in employee)  
  
False
```

The key "Department" does not exist, so Python returns False.

Section 6: Sets

What You Will Learn

In this section, you will learn:

- Creating sets
- Removing duplicates
- Adding elements
- Removing elements
- Set operations
- Why Is This Important for Data Analysts?

Sets are useful when you need:

- Unique values only
- Duplicate removal
- Comparing groups of data
- Finding common or missing values

#Q 51

```
numbers = {10, 20, 30}  
  
print(numbers)  
  
{10, 20, 30}
```

A set stores unique values inside curly braces {}.

#Q 52

```
numbers = {10, 20, 20, 30}
```

```
print(numbers)
```

```
{10, 20, 30}
```

Sets automatically remove duplicate values.

Even though 20 appears twice, it is stored only once.

#Q 53

```
numbers = {10, 20, 30}
```

```
numbers.add(40)
```

```
print(numbers)
```

```
{40, 10, 20, 30}
```

The order is not guaranteed.

#Q 54

```
numbers = {10, 20, 30}
```

```
numbers.remove(20)
```

```
print(numbers)
```

```
{10, 30}
```

Just like add() only works when the value doesn't already exist, remove() only works when the value does exist.

#Q 55

```
numbers = {10, 20, 30}
```

```
print(len(numbers))
```

```
3
```

len() returns the number of elements in the set.

#Q 56

```
numbers = {10, 20, 30}
```

```
print(20 in numbers)
```

True

#Q 57

```
numbers = {10, 20, 30}
```

```
print(50 in numbers)
```

False

Q58

```
set1 = {10, 20, 30}
```

```
set2 = {30, 40, 50}
```

```
print(set1.union(set2))
```

```
{50, 20, 40, 10, 30}
```

union() combines both sets and removes duplicates.

#Q 59

```
set1 = {10, 20, 30}
```

```
set2 = {30, 40, 50}
```

```
print(set1.intersection(set2))
```

```
{30}
```

intersection() returns values common to both sets.

30 exists in both sets.

#Q 60

```
set1 = {10, 20, 30}
```

```
set2 = {30, 40, 50}
```

```
print(set1.difference(set2))
```

```
{10, 20}
```

difference() returns values present in the first set but not in the second set.

10 and 20 exist only in set1.

Method	Purpose
add()	Add a value
remove()	Remove a value
len()	Count elements

Method	Purpose
<code>in</code>	Check existence
<code>union()</code>	Combine sets
<code>intersection()</code>	Common values
<code>difference()</code>	Unique values from first set

Section 7: Conditional Statements

What You Will Learn In this section, you will learn:

- `if`
- `else`
- `elif`
- Comparison operators
- Logical operators Decision making in Python
- Why Is This Important for Data Analysts? Conditional statements help Python make decisions.

Examples:

- If Sales > Target → Bonus
- If Age >= 18 → Eligible
- If Revenue < 0 → Data Error

This logic is used constantly in data cleaning and business analysis.

```
#Q 61
age = 20
if age >= 18:
    print("Eligible")
Eligible

#Q 62
age = 15
if age >= 18:
    print("Eligible")
```

Since the condition is false, Python skips the print statement.

```
#Q 63
age = 15
```

```
if age >= 18:  
    print("Eligible")  
else:  
    print("Not Eligible")
```

Not Eligible

```
#Q 64  
num = 10  
  
if num == 10:  
    print("Match")
```

Match

```
#Q 65  
num = 15  
  
if num != 10:  
    print("Different")
```

Different

```
#Q 66  
marks = 85  
  
if marks >= 90:  
    print("A")  
else:  
    print("B")
```

B

85 >= 90

is False.

So Python executes the else block.

```
#Q 67  
marks = 85  
  
if marks >= 90:  
    print("A")  
elif marks >= 80:  
    print("B")  
else:  
    print("C")
```

B

The second condition:

85 >= 80

True.

So Python prints:

B

```
#Q 68
age = 25

if age > 18 and age < 30:
    print("Young Adult")
```

Young Adult

25 > 18

and

25 < 30

Since both are true, and returns True.

```
#Q 69
age = 35

if age < 18 or age > 30:
    print("Outside Range")
```

Outside Range

or requires only one condition to be true.

35 > 30

is true.

Therefore Python executes the print statement.

```
# Q70
is_active = False

if not is_active:
    print("Inactive")
```

Inactive

not False becomes True.

Therefore the print statement executes.

```
# Q71
salary = 50000

if salary >= 50000:
    print("Eligible for Loan")
```

Eligible for Loan

```
#Q 72
salary = 45000

if salary >= 50000:
    print("Eligible for Loan")
else:
    print("Not Eligible")
```

Not Eligible

```
#Q 73
num = 8

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Even

8 % 2 gives:

0

Since: 0 == 0 is True, Python prints:

Even

```
# Q74
num = 7

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Odd

7 % 2 gives:

1 Since:

1 == 0 is False, Python executes the else block.

```
# Q75
age = 25

if age >= 18:
    if age >= 21:
        print("Can Drive and Drink")
```

Can Drive and Drink

25 >= 18

is True.

Second condition:

25 >= 21

is also True.

Since both conditions are true, Python prints:

Can Drive and Drink

Section 8: Loops

What You Will Learn

- for loops
- while loops
- range()
- break
- continue

Why Is This Important for Data Analysts?

- Loops help automate repetitive tasks.

Instead of writing:

```
print(1), print(2), print(3), print(4), print(5)
```

- you can use a loop to do it automatically.

This is a foundational concept before moving to Pandas and data processing.

```
# Q76
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

range(5) generates numbers from 0 to 4.

```
# Q77
for i in range(1, 6):
    print(i)
```

```
1
2
3
4
5
```

range(1, 6) starts at 1 and stops before 6.

```
# Q78
for i in range(2, 11, 2):
    print(i)
```

```
2
4
6
8
10
```

The third value (2) is the step size.

Python increases by 2 each time.

```
# Q79
fruits = ["Apple", "Banana", "Mango"]

for fruit in fruits:
    print(fruit)
```

```
Apple
Banana
Mango
```

The loop visits each item in the list one by one.

```
# Q80
for i in range(3):
    print("Python")
```

```
Python
Python
Python
```

The loop runs 3 times.

```
# Q81
i = 1

while i <= 3:
    print(i)
    i += 1
```

```
1
2
3
```

The loop continues while $i \leq 3$.

After each iteration, i increases by 1.

```
# Q 82

i = 5

while i > 0:
    print(i)
    i -= 1
```

```
5
4
3
2
1
```

The loop counts backward from 5 to 1.

```
# Q83
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
0
1
2
```

break immediately stops the loop when i becomes 3.

```
# Q84
for i in range(5):
    if i == 3:
        continue
    print(i)

0
1
2
4
```

continue skips the current iteration.

When i is 3, Python skips print(i) and moves to the next iteration.

```
# Q85
for i in range(1, 4):
    print(i * 2)

2
4
6
```

Each value is multiplied by 2.

```
# Q86
total = 0

for i in range(1, 4):
    total = total + i

print(total)

6

# Q87
for i in range(1, 6):
    if i % 2 == 0:
        print(i)

2
4
```

Only even numbers satisfy:

$i \% 2 == 0$

Among 1, 2, 3, 4, 5, the even numbers are:

24

```
# Q88
for i in range(1, 6):
    if i % 2 != 0:
        print(i)

1
3
5
```

`!= 0` means the remainder is not zero.

Only odd numbers satisfy this condition.

```
# Q89
word = "DATA"

for letter in word:
    print(letter)

D
A
T
A
```

A string is a collection of characters.

The loop visits one character at a time.

```
# Q90
for i in range(1, 4):
    print(i)
else:
    print("Done")

1
2
3
Done
```

The loop completes normally.

After the loop finishes, the else block executes.

Section 9: Functions (Q91–Q105)

What You Will Learn

- Creating functions

- Function parameters
- Return values *Reusable code

Why Is This Important for Data Analysts?

Imagine calculating profit in 20 different places:

- $\text{profit} = \text{revenue} - \text{cost}$

Instead of writing it repeatedly, you can create a function and reuse it anywhere.

Functions make code:

- Cleaner
- Reusable
- Easier to maintain

```
# Q91
def greet():
    print("Hello")

greet()

Hello
```

`greet()` is a function. When called, it executes the code inside it.

```
# Q92
def greet():
    print("Hello")

greet()
greet()

Hello
Hello
```

The function is called twice, so it executes twice.

```
# Q93
def greet(name):
    print("Hello", name)

greet("Shubham")
```

```
Hello Shubham
```

name receives the value "Shubham".

```
# Q94
def add(a, b):
    print(a + b)

add(10, 20)
30

# Q95
def multiply(a, b):
    print(a * b)

multiply(5, 4)
20

# Q96
def square(num):
    print(num **2)
square(5)
25

# Q97
def greet(name):
    return "Hello " + name

print(greet("Shubham"))
Hello Shubham
```

return sends the value back to where the function was called.

```
# Q98
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
30
```

$10 + 20 = 30$

which gets stored in result.

```
#Q 99

def greet():
    return "Hello"
message = greet()

print(message)

Hello
```

The returned value is stored in message and then printed.

```
#Q 100

def profit(revenue, cost):
    return revenue - cost

print(profit(1000, 700))

300
```

$1000 - 700 = 300$

The function returns the profit.

There is a big difference between:

print()

and

return

print() displays output.

return sends a value back from the function.

Section 10: Advanced Functions

What You Will Learn

- Multiple return values
- Default parameters
- Variable scope
- Lambda functions
- Built-in functions useful for Data Analysts

Why Is This Important for Data Analysts?

As projects grow, functions help organize logic into reusable blocks.

For example:

- Calculate KPIs
- Clean data
- Transform columns
- Automate repetitive tasks

```
#Q 101
def add(a,b):
    return a + b

print(add(5,10))
15
```

The function receives:

a = 5 b = 10

and returns:

5 + 10 = 15

```
#Q 102
def square(num):
    return num * num

result = square(4)
print(result)
16
```

4 * 4 = 16

The returned value is stored in result.

```
#Q 103
def greet(name = "Guest"):
    print("Hello", name)

greet()
Hello Guest
```

#Q 104

```
def greet(name = "Guest"):  
    print("Hello", name)
```

```
greet ("Shubham")
```

Hello Shubham

#Q 105

```
def calc(a,b):  
    return a + b, a * b  
print(calc(2,3))
```

(5, 6)

Python returns two values:

$2 + 3 = 5$

$2 * 3 = 6$

These values are returned as a tuple.

106

```
x = 10  
def show():  
    print(x)  
show()
```

10

The function can access variables defined outside the function.

#Q 107

```
def show():  
    x = 20  
    print(x)
```

```
show()
```

20

x is created inside the function and exists only within that function.

#Q 108

```
square = lambda x: x ** 2  
print(square(5))
```

25

lambda creates a small anonymous function.

$5 ** 2 = 25$

#Q 109

```
numbers = [10,20,30]  
print(max(numbers))
```

30

max() returns the largest value in a collection.

#Q 110

```
numbers = [10,20,30]  
print(sum(numbers))
```

60

sum() adds all elements together.

$10 + 20 + 30 = 60$

Section 11: Useful Built-in Functions

What You Will Learn

- min()
- max()
- sum()
- len()
- type()
- sorted()
- abs()
- round()

Why Is This Important for Data Analysts?

These functions are used constantly when:

- Exploring datasets
- Checking data quality
- Creating summary statistics
- Performing calculations

Q111

```
numbers = [10,20,30]
print(min(numbers))

10
```

Why?

min() returns the smallest value in the collection.

Q112

```
numbers = [10,20,30,40]
print(max(numbers))

40
```

Why?

max() returns the largest value in the collection.

#Q 113

```
numbers = [10,20,30]
print(sum(numbers))

60
```

Why?

sum() adds all values together.

Q114

```
numbers = [10,20,30]
print(len(numbers))

3
```

Why?

len() counts the number of elements.

Q115

```
x =10
print(type(x))
```



```
<class 'int'>
```

Why?

type() tells us the datatype of a variable.

```
# Q116
x = "Python"
print(type(x))
<class 'str'>
```

Why?

The value is enclosed in quotes, making it a string (str).

```
# Q117
numbers = [30,10,20]
print(sorted(numbers))
[10, 20, 30]
```

Why?

sorted() returns a sorted version of the list.

```
# Q117A
numbers = [30,10,20]
print(sorted(numbers, reverse =True))
[30, 20, 10]
# Q117B
numbers = [30, 10, 20]
numbers.sort()
print(numbers)
[10, 20, 30]
```

sort() modifies the original list.

```
#117c
numbers = [30, 10, 20]
numbers.sort(reverse=True)
```

```
print(numbers)
[30, 20, 10]
```

Why?

sort(reverse=True) sorts the original list in descending order.

```
numbers = [30, 10, 20]

print(sorted(numbers))
print(numbers)

[10, 20, 30]
[30, 10, 20]

# sort()

# Changes the original list itself.

numbers = [30, 10, 20]

numbers.sort()

print(numbers)

[10, 20, 30]

# Q118
print(abs(-25))

25
```

Why?

abs() returns the absolute (positive) value.

```
# Q119
print(round(3.66))

4
```

round() rounds the number to the nearest whole number.

```
# Q120
print(round(3.14159, 2))

3.14
```

Why?

The second argument specifies the number of decimal places.

Section 12: List Comprehensions & Intermediate Python

What You Will Learn

- List Comprehensions
- Creating Lists Efficiently
- Filtering Data
- Conditional Logic in Lists

Why Is This Important for Data Analysts?

List comprehensions are heavily used for:

- Data Cleaning
- Data Transformation
- Feature Creation
- Preparing data before using Pandas

```
#Q 121
numbers = [x for x in range(5)]
print(numbers)

[0, 1, 2, 3, 4]
```

The list comprehension collects these values into a list.

```
#Q 122
numbers = [x * 2 for x in range(5)]
print(numbers)

[0, 2, 4, 6, 8]
```

Why?

Each value generated by range(5) is multiplied by 2 before being added to the list.

```
# Q123
numbers = [ x for x in range (1,6) if x % 2 == 0]
print(numbers)

[2, 4]
```

Why?

Only even numbers satisfy:

$x \% 2 == 0$

Q124

```
numbers = [x for x in range(1,6) if x%2 != 0]
print(numbers)
[1, 3, 5]
```

Why?

Only odd numbers satisfy:

$x \% 2 != 0$

Q 125

```
names = ["Shubham", "Anant", "Sourabh"]
lengths = [len(name) for name in names]
print(lengths)
[7, 5, 7]
```

Why?

Python calculates the length of each string and stores the results in a new list.

#Q 126

```
names = ["Shubham", "Anant", "Sourabh"]
result = [name.upper() for name in names]
print(result)
['SHUBHAM', 'ANANT', 'SOURABH']
```

Why?

.upper() converts each string to uppercase before adding it to the new list.

#Q 127

```
numbers = [10, 20, 30]
result = [num + 5 for num in numbers]
print(result)
```

```
[15, 25, 35]
```

Why?

Each element is increased by 5.

```
#Q 128
numbers = [1, 2, 3, 4]

result = [num ** 2 for num in numbers]

print(result)

[1, 4, 9, 16]
```

Each number is squared before being added to the new list.

```
#Q 129

text = "DATA"

result = [char for char in text]

print(result)

['D', 'A', 'T', 'A']
```

Why?

A string is iterable, so Python processes one character at a time.

```
numbers = [10, 20, 30]

result = [num for num in numbers]

print(result)

[10, 20, 30]
```

Why?

Python copies each element from the original list into a new list.

Section 13: Advanced Conditional Statements

What You Will Learn

- Nested if

- Multiple Conditions
- Real-World Business Logic
- Decision Trees
- Combining if, elif, and else

Why Is This Important for Data Analysts?

Most business rules are not simple.

Instead of:

```
if age >= 18: print("Eligible")
```

you'll often see:

```
if sales >= target: if customer_satisfaction >= 90: print("Bonus")
```

-- Real-world analytics is full of nested conditions.

```
# Q131
```

```
age = 25
if age >= 18:
    if age >= 21:
        print("Adult")
```

```
Adult
```

```
#Q 132
```

```
age = 19

if age >= 18:
    if age >= 21:
        print("Adult")
```

```
# Q 133
```

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("D")
```

B

```
#Q 134
salary = 60000

if salary >= 50000 and salary <= 100000:
    print("Eligible")

Eligible

#Q 135
salary = 120000

if salary >= 50000 and salary <= 100000:
    print("Eligible")
else:
    print("Not Eligible")

Not Eligible
```

The second condition is false:

120000 <= 100000

```
#Q 136

city = "Mumbai"

if city == "Mumbai" or city == "Delhi":
    print("Metro City")

Metro City
```

Why?

or requires only one condition to be true.

```
#Q 137
is_member = True

if not is_member:
    print("Guest")
else:
    print("Member")

Member
```

Why?

not True becomes False.

Therefore Python executes the else block.

#Q 138

```
sales = 120000

if sales >= 100000:
    print("Target Achieved")
else:
    print("Target Missed")
```

Target Achieved

Q139

```
sales = 120000

if sales >= 150000:
    print("Gold Bonus")
elif sales >= 100000:
    print("Silver Bonus")
else:
    print("No Bonus")
```

Silver Bonus

Why?

The first condition is false.

The second condition is true.

#Q 140

```
sales = 180000

if sales >= 100000:
    if sales >= 150000:
        print("Gold Bonus")
    else:
        print("Silver Bonus")
```

Gold Bonus

Rule to Remember

- Use if-elif-else

When only one outcome should happen.

Example:

- Grade Classification
- Bonus Category
- Customer Segment

One-Line Memory Trick

- if-elif-else → First True wins.

Multiple if statements → Check everything.

- This distinction is extremely important and comes up often in Python interviews.

Section 14: Advanced Loops

What You Will Learn

- Nested Loops
- Looping Through Dictionaries
- Looping Through Tuples
- Looping Through Sets
- enumerate()
- zip()
- Multiplication Tables
- Pattern Building

Why Is This Important for Data Analysts?

In real projects, you'll rarely write:

for i in range(5):

```
print(i)
```

Instead you'll be:

- Looping through records
- Processing key-value pairs
- Combining multiple columns
- Generating reports
- Working with complex data structures

#Q141

```
for i in range(2):  
    for j in range(2):  
        print(i, j)
```

```
0 0  
0 1  
1 0  
1 1
```

Why?

For every value of i, the inner loop runs completely.

#Q 142

```
for i in range(1, 4):  
    print(i * 5)
```

```
5  
10  
15
```

Why?

Each value generated by the loop is multiplied by 5.

#Q 143

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
for key in employee:  
    print(key)
```

```
Name  
Age
```

Why?

By default, looping through a dictionary returns the keys.

#Q 144

```
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}
```

```
for value in employee.values():  
    print(value)
```

```
Shubham  
28
```

Why?

.values() returns all values from the dictionary.

```
#Q 145  
employee = {  
    "Name": "Shubham",  
    "Age": 28  
}  
  
for key, value in employee.items():  
    print(key, value)
```

```
Name Shubham  
Age 28
```

Why?

.items() returns both the key and value.

```
#Q 146  
numbers = (10, 20, 30)  
  
for num in numbers:  
    print(num)
```

```
10  
20  
30
```

Tuples are iterable just like lists.

```
#Q 147  
numbers = {10, 20, 30}  
  
for num in numbers:  
    print(num)
```

```
10  
20  
30
```

Why?

The loop visits each value in the set.

Note: Order is not guaranteed.

```
#Q 148
names = ["Amit", "Rahul", "Shubham"]

for index, name in enumerate(names):
    print(index, name)

0 Amit
1 Rahul
2 Shubham
```

enumerate() provides both the index and the value.

```
#Q 149

names = ["Amit", "Rahul"]
scores = [80, 90]

for name, score in zip(names, scores):
    print(name, score)

Amit 80
Rahul 90
```

zip() combines elements from multiple lists.

```
#Q 150

for i in range(1, 4):
    print("*" * i)

*
**
***
```

The string "*" is repeated i times during each iteration.

Section 15: Advanced String Operations

What You Will Learn

- split()
- join()
- replace()
- strip()

- find()
- startswith()
- endswith()

Why Is This Important for Data Analysts?

A huge part of analytics is cleaning messy text data.

```
#Q151
text = "Python Data Analyst"

print(text.split())

['Python', 'Data', 'Analyst']
```

Why?

split() breaks a string into a list.

By default it splits using spaces.

```
#Q152
text = "Apple,Banana,Mango"

print(text.split(","))

['Apple', 'Banana', 'Mango']
```

Why?

Python splits the string wherever it finds a comma.

```
#Q153

fruits = ['Apple', 'Banana', 'Mango']

print("-".join(fruits))

Apple-Banana-Mango
```

Why?

join() combines list elements into a single string using the specified separator.

```
#Q 154

text = "I love Python"

print(text.replace("Python", "SQL"))
```

I love SQL

Why?

replace(old, new) replaces the specified text.

```
#Q 155
text = "Data Analyst"
print(text.replace("a", "A"))
DAtA AnAlyst
```

Why?

Every lowercase "a" is replaced with uppercase "A".

Python is case-sensitive.

```
#Q 156
text = "  Python  "
print(text.strip())
Python
```

Why?

strip() removes spaces from both the beginning and end of a string.

```
#Q157
text = "Python Data Analyst"
print(text.find("Data"))
7

#Q 158
text = "Python Data Analyst"
print(text.find("SQL"))
-1
```

Why?

"SQL" does not exist in the string.

find() returns -1 when the text is not found.

#Q 159

```
text = "Python Data Analyst"  
  
print(text.startswith("Python"))
```

True

Why?

The string begins with "Python".

#Q160

```
text = "Python Data Analyst"  
  
print(text.endswith("Analyst"))
```

True

Why?

The string ends with "Analyst".

Conclusion

This question bank provided a structured approach to learning Python fundamentals through hands-on practice and logical reasoning.

The focus was not only on writing code but also on understanding how Python executes instructions, manipulates data, and solves problems. Through progressive exercises covering core programming concepts and data structures, a strong foundation was established for analytical thinking and data-driven problem solving.

The concepts covered in this notebook form the building blocks required for applying Python in data analytics, data manipulation, and exploratory analysis.